A Student-Friendly Summary of AFDPS for a Navier–Stokes Inverse Problem

1 Big Picture

This note explains how a modern posterior-sampling algorithm, called AFDPS, can be applied to an inverse problem involving the two-dimensional Navier–Stokes equation. The goal is to make the mathematical and computational ideas accessible to a motivated high school student with some background in calculus, linear algebra, and basic probability.

The main question is:

> Given partial and noisy measurements of a fluid at a later time, can we infer what the fluid looked like at the beginning?

This is an inverse problem because we observe an effect and try to recover its cause.

2 What is the Navier–Stokes Equation?

The Navier–Stokes equation is one of the fundamental equations of fluid mechanics. It describes how fluids such as water or air move over time.

In two dimensions, instead of tracking the velocity directly, it is often useful to track the *vorticity*. Vorticity measures local spinning motion in the fluid. For example, a small whirlpool has high vorticity.

We work on a periodic square domain

$$\mathbb{T}^2 = [0, 2\pi]^2,$$

which means that the left edge is connected to the right edge, and the top edge is connected to the bottom edge. This is mathematically convenient and allows the use of Fourier methods.

The two-dimensional Navier–Stokes equation in vorticity form is

$$\partial_t \omega + u \cdot \nabla \omega = \nu \Delta \omega + f. \tag{1}$$

Here:

- $\omega(x, t)$ is the vorticity field;
- $u(x, t)$ is the velocity field;
- ν is viscosity, which smooths the fluid motion;
- f is an external force;
- Δ is the Laplacian, a second-derivative operator;
- $u \cdot \nabla \omega$ is the nonlinear transport term.

The velocity u is recovered from ω by solving

$$u = \nabla^{\perp} \psi, \qquad -\Delta \psi = \omega. \tag{2}$$

This means the vorticity determines the velocity through a Poisson equation.

3 The Forward Problem

Suppose we know the initial vorticity field

$$\omega_0(x) = \omega(x, 0).$$

The forward problem is to solve Navier–Stokes and compute the final-time vorticity

$$\omega(T).$$

We write this solution operator as

$$\mathcal{L}(\omega_0) = \omega(T). \tag{3}$$

The operator $\mathcal{L}$ is nonlinear because the Navier–Stokes equation contains the nonlinear term

$$u \cdot \nabla \omega.$$

4 The Inverse Problem

In the inverse problem, we do not know ω_0. Instead, we observe only part of the final state:

$$y = P\mathcal{L}(\omega_0) + n. \tag{4}$$

Here:

- y is the observed data;
- P is a measurement operator, for example downsampling or observing only some pixels;
- n is Gaussian noise;
- ω_0 is the unknown initial vorticity field.

The task is to infer

$$\omega_0$$

from the partial noisy observation y.

5 Bayesian Viewpoint

Bayesian inference combines two sources of information:

1. a prior, which describes what we believe about ω_0 before seeing data;
2. a likelihood, which describes how likely the observed data y is for each possible ω_0.

The posterior distribution is

$$p(\omega_0 \mid y) \propto p_0(\omega_0)\, p(y \mid \omega_0). \tag{5}$$

Equivalently,

$$p(\omega_0 \mid y) \propto p_0(\omega_0) \exp\bigl(-\mu_y(\omega_0)\bigr), \tag{6}$$

where

$$\mu_y(\omega_0) = \frac{1}{2\sigma^2} \|P\mathcal{L}(\omega_0) - y\|^2. \tag{7}$$

The function μ_y is the negative log-likelihood.

6 The Prior is Gaussian in This Benchmark

In the synthetic Navier–Stokes benchmark, the initial vorticity is sampled from a known Gaussian random field:

$$\omega_0 \sim \mathcal{N}\left(0, (-\Delta + 9I)^{-4}\right). \tag{8}$$

Thus the true prior is analytically known. Let

$$C = (-\Delta + 9I)^{-4}.$$

Then

$$p_0(\omega_0) \propto \exp\left(-\frac{1}{2}\langle \omega_0, C^{-1}\omega_0\rangle\right). \tag{9}$$

The prior score is

$$\nabla_{\omega_0} \log p_0(\omega_0) = -C^{-1}\omega_0 = -(-\Delta + 9I)^4\omega_0. \tag{10}$$

In Fourier space, if

$$\omega_0(x) = \sum_k \widehat{\omega}_{0,k} e^{ik\cdot x},$$

then

$$\widehat{C^{-1}\omega_0}_k = (|k|^2 + 9)^4 \widehat{\omega}_{0,k}. \tag{11}$$

This is important: for this particular benchmark, one does not strictly need to train a neural network prior, because the true Gaussian prior is already available.

7 What is AFDPS?

AFDPS stands for an accelerated or adjusted form of diffusion posterior sampling. The broad idea is to use a generative sampling method to draw samples from the posterior distribution

$$p(\omega_0 \mid y).$$

Instead of producing just one best guess, posterior sampling produces many plausible solutions. This is useful because inverse problems are often uncertain: many different initial fluid states may produce similar observations.

AFDPS combines:

- a prior score, which tells the sampler what typical samples look like;
- likelihood information, which tells the sampler how well a sample explains the observed data;
- a correction term involving the Laplacian of the likelihood.

For this problem, the prior score can be computed exactly from the Gaussian prior. The main computational challenge is evaluating the likelihood gradient

$$\nabla_{\omega_0}\mu_y(\omega_0).$$

8 AFDPS Algorithm: Clear Explanation and Outline

AFDPS is a posterior-sampling method. Its goal is to generate samples from

$$p(x \mid y) \propto p_0(x) \exp(-\mu_y(x)), \tag{12}$$

where x is the unknown variable, y is the observation, p_0 is the prior distribution, and μ_y is the negative log-likelihood.

For the Navier–Stokes inverse problem,

$$x = \omega_0, \tag{13}$$

so the goal is to sample plausible initial vorticity fields conditioned on the observed final-time data.

8.1 Conceptual Explanation

AFDPS combines three pieces of information:

1. **Prior information:** the sample should look like a plausible initial vorticity field.
2. **Data information:** after solving Navier–Stokes forward, the sample should match the observation y.
3. **Randomness:** the sampler should explore many possible solutions, not just one best reconstruction.

The prior information is represented by the prior score

$$\nabla_x \log p_0(x). \tag{14}$$

The data information is represented by the likelihood gradient

$$\nabla_x \mu_y(x). \tag{15}$$

AFDPS may also use a correction involving the likelihood Laplacian

$$\Delta_x \mu_y(x) = \mathrm{Tr}(\nabla_x^2 \mu_y(x)). \tag{16}$$

8.2 Specialization to Navier–Stokes

For this benchmark, the posterior is

$$p(\omega_0 \mid y) \propto \exp\left(-\frac{1}{2}\langle \omega_0, C^{-1}\omega_0\rangle - \frac{1}{2\sigma^2}\|P\mathcal{L}(\omega_0) - y\|^2\right), \tag{17}$$

where

$$C = (-\Delta + 9I)^{-4}. \tag{18}$$

Thus AFDPS needs:

- the Gaussian prior score

$$\nabla_{\omega_0} \log p_0(\omega_0) = -(-\Delta + 9I)^4 \omega_0; \tag{19}$$

- the likelihood gradient

$$\nabla_{\omega_0} \mu_y(\omega_0); \tag{20}$$

- the likelihood Laplacian estimate

$$\Delta_{\omega_0} \mu_y(\omega_0). \tag{21}$$

The prior score is cheap because it is diagonal in Fourier space. The likelihood gradient is expensive because it requires a forward Navier–Stokes solve and a backward adjoint solve.

8.3 Step-by-Step Algorithm

1. **Initialize particles.** Start with J candidate initial vorticity fields

$$\omega_0^{(1)}, \dots, \omega_0^{(J)}. \tag{22}$$

2. **Compute the prior score.** For each particle, compute

$$s_{\text{prior}}^{(j)} = -(-\Delta + 9I)^4 \omega_0^{(j)}. \tag{23}$$

3. **Solve the forward Navier–Stokes equation.** Starting from $\omega_0^{(j)}$, solve forward to obtain

$$\omega^{(j)}(T) = \mathcal{L}(\omega_0^{(j)}). \tag{24}$$

4. **Compute the residual.** Compare the predicted observation with the actual observation:

$$r^{(j)} = P\omega^{(j)}(T) - y. \tag{25}$$

5. **Solve the adjoint equation.** Set

$$\lambda^{(j)}(T) = \frac{1}{\sigma^2} P^* r^{(j)}. \tag{26}$$

   Then solve backward in time:

$$-\partial_t \lambda = \nu\Delta\lambda + u \cdot \nabla\lambda + (-\Delta)^{-1}\nabla^\perp \cdot (\lambda \nabla\omega). \tag{27}$$

   The likelihood gradient is

$$\nabla_{\omega_0} \mu_y(\omega_0^{(j)}) = \lambda^{(j)}(0). \tag{28}$$

6. **Estimate the likelihood Laplacian.** Use Hutchinson trace estimation:

$$\Delta\mu_y(\omega_0) \approx \frac{1}{M} \sum_{m=1}^{M} \frac{\langle \nabla\mu_y(\omega_0 + \varepsilon\xi_m) - \nabla\mu_y(\omega_0 - \varepsilon\xi_m), \xi_m \rangle}{2\varepsilon}. \tag{29}$$

7. **Update the particle.** Use the AFDPS update rule, combining the prior score, likelihood gradient, likelihood Laplacian estimate, and Gaussian noise.

8. **Repeat.** Iterate until the particles approximately follow the posterior distribution $p(\omega_0 \mid y)$.

8.4 Simplified Pseudocode

```
Input: observation y, number of particles J, number of AFDPS steps K

Initialize particles omega0[1], ..., omega0[J]

for k = 1,...,K:
    for each particle j:
        prior_score = -(-Delta + 9I)^4 omega0[j]

        solve forward Navier--Stokes from omega0[j] to omega_T[j]
        residual = P omega_T[j] - y
```

```
        solve adjoint Navier--Stokes backward from terminal residual
        likelihood_grad = lambda(0)

        estimate Delta_mu using Hutchinson finite differences

        omega0[j] = AFDPS_update(
            omega0[j], prior_score, likelihood_grad, Delta_mu, noise
        )

Return posterior samples omega0[1], ..., omega0[J]
```

8.5 Practical Notes

- The expensive part is solving the forward and adjoint PDEs.
- The prior score is cheap because the Gaussian prior is diagonal in Fourier space.
- The adjoint gradient should be verified by finite-difference checks.
- The Hutchinson estimate can be noisy, so one may start with $M = 1$ or $M = 2$.
- Multiple particles can be solved in parallel on a GPU.

9 Why the Likelihood Gradient is Hard

The likelihood is

$$\mu_y(\omega_0) = \frac{1}{2\sigma^2}\|P\mathcal{L}(\omega_0) - y\|^2. \tag{30}$$

The operator $\mathcal{L}$ is not a matrix. It means: solve the nonlinear Navier–Stokes equation from time 0 to time T.

By the chain rule,

$$\nabla_{\omega_0}\mu_y(\omega_0) = D\mathcal{L}(\omega_0)^* P^* \frac{P\mathcal{L}(\omega_0) - y}{\sigma^2}. \tag{31}$$

Computing this gradient efficiently requires an adjoint PDE.

10 The Adjoint Equation

First solve the forward Navier–Stokes equation from 0 to T to obtain $\omega(t)$ and $u(t)$.

Then define the terminal condition

$$\lambda(T) = \frac{1}{\sigma^2}P^*(P\omega(T) - y). \tag{32}$$

The adjoint equation is solved backward in time:

$$-\partial_t\lambda = \nu\Delta\lambda + u\cdot\nabla\lambda + (-\Delta)^{-1}\nabla^\perp\cdot(\lambda\nabla\omega). \tag{33}$$

The likelihood gradient is

$$\boxed{\nabla_{\omega_0}\mu_y(\omega_0) = \lambda(0).} \tag{34}$$

Thus, each gradient evaluation requires:

1. one forward Navier–Stokes solve;
2. one backward adjoint solve.

11 What are Spectral and Pseudo-Spectral Methods?

A spectral method represents a function using Fourier waves. For example,

$$\omega(x,t) = \sum_k \widehat{\omega}_k(t) e^{ik\cdot x}. \tag{35}$$

On a periodic domain, derivatives are especially simple in Fourier space:

$$\widehat{\partial_x f}_k = ik_x \widehat{f}_k, \qquad \widehat{\Delta f}_k = -|k|^2 \widehat{f}_k. \tag{36}$$

A pseudo-spectral method uses both Fourier space and physical grid space:

- derivatives and Poisson solves are computed in Fourier space;
- nonlinear products are computed pointwise in physical space;
- FFTs are used to move between the two.

This is efficient because the Navier–Stokes equation contains both linear differential operators and nonlinear products.

12 Handling the Inverse Laplacian

The adjoint equation contains the term

$$(-\Delta)^{-1}\nabla^\perp \cdot (\lambda \nabla \omega). \tag{37}$$

Let

$$q = \lambda \nabla \omega.$$

Then

$$q_1 = \lambda \partial_x \omega, \qquad q_2 = \lambda \partial_y \omega.$$

We compute

$$g = \nabla^\perp \cdot q = -\partial_y q_1 + \partial_x q_2. \tag{38}$$

In Fourier space,

$$\widehat{g}_k = -ik_y \widehat{q}_{1,k} + ik_x \widehat{q}_{2,k}. \tag{39}$$

Then

$$\widehat{(-\Delta)^{-1} g}_k = \frac{\widehat{g}_k}{|k|^2}, \qquad k \neq 0. \tag{40}$$

For the zero Fourier mode, set

$$\widehat{(-\Delta)^{-1} g}_0 = 0. \tag{41}$$

This zero-mode convention fixes the arbitrary additive constant in the Poisson solution.

13 Forward Pseudo-Spectral Solver

At each time step:

1. Start with ω^n.
2. Use FFT to compute derivatives and solve $-\Delta\psi = \omega^n$.
3. Recover velocity $u^n = \nabla^\perp \psi^n$.

4. Compute $u^n \cdot \nabla \omega^n$ in physical space.

5. Advance the PDE in time, often treating viscosity semi-implicitly.

A typical semi-implicit update is

$$\widehat{\omega}_k^{n+1} = \frac{\widehat{\omega}_k^n + \Delta t \left[-\widehat{u^n \cdot \nabla \omega^n}_k + \widehat{f}_k \right]}{1 + \Delta t \, \nu |k|^2}. \tag{42}$$

14 Backward Adjoint Pseudo-Spectral Solver

Starting from

$$\lambda^N = \frac{1}{\sigma^2} P^*(P\omega^N - y),$$

we march backward.

At each backward step:

1. Load stored forward fields ω^n and u^n.

2. Compute $\nabla \lambda^n$ spectrally.

3. Compute $u^n \cdot \nabla \lambda^n$ in physical space.

4. Compute

$$a^n = (-\Delta)^{-1} \nabla^\perp \cdot (\lambda^n \nabla \omega^n).$$

5. Update λ backward in time.

A semi-implicit update is

$$\widehat{\lambda}_k^{n-1} = \frac{\widehat{\lambda}_k^n + \Delta t \, \widehat{B}_k^n}{1 + \Delta t \, \nu |k|^2}, \tag{43}$$

where

$$B^n = u^n \cdot \nabla \lambda^n + a^n. \tag{44}$$

Finally,

$$\nabla_{\omega_0} \mu_y = \lambda^0. \tag{45}$$

15 Likelihood Laplacian and Hutchinson Estimation

AFDPS may also require the Laplacian of the likelihood:

$$\Delta \mu_y(\omega_0) = \mathrm{Tr}\left(\nabla^2 \mu_y(\omega_0) \right). \tag{46}$$

Deriving a second-order adjoint equation is possible but complicated. Instead, we can use Hutchinson trace estimation.

Choose random vectors ξ_m with entries ± 1. Then

$$\Delta \mu_y(\omega_0) \approx \frac{1}{M} \sum_{m=1}^{M} \xi_m^\top \nabla^2 \mu_y(\omega_0) \xi_m. \tag{47}$$

Each term can be approximated using finite differences of gradients:

$$\xi^\top \nabla^2 \mu_y(\omega_0) \xi \approx \frac{\langle \nabla \mu_y(\omega_0 + \varepsilon \xi) - \nabla \mu_y(\omega_0 - \varepsilon \xi), \xi \rangle}{2\varepsilon}. \tag{48}$$

This only requires the first-order adjoint gradient routine.

16 Complete Research Plan

A possible student research project could proceed as follows.

1. Study the 2D vorticity form of Navier–Stokes.
2. Implement a pseudo-spectral forward solver.
3. Verify the solver on simple initial conditions.
4. Implement the adjoint solver.
5. Check the adjoint gradient using finite differences:

$$\frac{\mu(\omega_0 + \varepsilon h) - \mu(\omega_0)}{\varepsilon} \approx \langle \nabla \mu(\omega_0), h \rangle.$$

6. Implement the Gaussian prior score exactly.
7. Implement Hutchinson trace estimation.
8. Combine these pieces inside an AFDPS sampler.
9. Compare posterior samples with the true initial condition.

17 Main Takeaways

- The Navier–Stokes equation models fluid motion.
- The inverse problem asks us to infer the initial fluid state from later partial observations.
- The benchmark prior is a known Gaussian random field, so training a prior neural network is not strictly necessary.
- The main difficulty is the nonlinear Navier–Stokes likelihood.
- The likelihood gradient can be computed with an adjoint PDE.
- The likelihood Laplacian can be estimated using Hutchinson trace estimation.
- Pseudo-spectral methods are natural because the domain is periodic.

18 Useful References and Resources

This project combines ideas from fluid dynamics, inverse problems, Bayesian inference, scientific computing, and diffusion-based generative modeling. The following references and software resources are useful starting points.

18.1 Introductory References on Navier–Stokes and Fluid Dynamics

1. A. J. Chorin and J. E. Marsden, *A Mathematical Introduction to Fluid Mechanics.* A classical introduction to incompressible fluid equations.
2. P. Constantin and C. Foias, *Navier–Stokes Equations.* A more advanced mathematical treatment.
3. Steven H. Strogatz, *Nonlinear Dynamics and Chaos.* Helpful background for dynamical systems and nonlinear PDE intuition.

18.2 Spectral Methods and Numerical PDEs

1. Lloyd N. Trefethen, *Spectral Methods in MATLAB*. A very accessible introduction to spectral methods.
2. John P. Boyd, *Chebyshev and Fourier Spectral Methods*. A standard reference for Fourier spectral PDE solvers.
3. Randall J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations*. Good numerical PDE background.

18.3 Inverse Problems and Bayesian Methods

1. Andrew M. Stuart, *Inverse problems: a Bayesian perspective*, Acta Numerica (2010).
2. M. Dashti and A. M. Stuart, *The Bayesian Approach to Inverse Problems*. A standard modern reference.

18.4 Diffusion Models and Posterior Sampling

1. Yang Song et al., *Score-Based Generative Modeling through Stochastic Differential Equations*.
2. Jonathan Ho et al., *Denoising Diffusion Probabilistic Models*.
3. The AFDPS paper associated with this project.

18.5 Useful GitHub Repositories

- InverseBench: `https://github.com/devzhk/InverseBench`

  Contains Navier–Stokes inverse-problem benchmarks and pretrained diffusion priors.
- JAX-CFD: `https://github.com/google/jax-cfd`

  A modern GPU-based differentiable fluid solver framework.
- PhiFlow: `https://github.com/tum-pbs/PhiFlow`

  A differentiable PDE and fluid simulation library.
- Dedalus: `https://github.com/DedalusProject/dedalus`

  A spectral PDE framework widely used in computational fluid dynamics.
- PyTorch FFT documentation: `https://pytorch.org/docs/stable/fft.html`

  Useful for implementing pseudo-spectral solvers.

19 Conclusion

This problem is a useful bridge between classical applied mathematics and modern generative modeling. Classical tools such as Fourier methods, adjoint PDEs, and Gaussian priors combine naturally with modern posterior sampling methods such as AFDPS.

For this benchmark, the key implementation ingredients are:

- a pseudo-spectral Navier–Stokes solver;
- a pseudo-spectral adjoint solver;

- an exact Gaussian prior score;
- Hutchinson trace estimation;
- an AFDPS posterior sampler.